

KUBERNATES

Today Task Overview & Importance (Summary)

Today I worked on **setting up and practicing Kubernetes cluster deployment** in both **local and cloud environments**. First, I configured **Minikube on my local machine**, which helped me understand how Kubernetes works in a single-node local setup for learning and testing.

Then I created a **real Kubernetes cluster on Ubuntu** with **1 Master (Control Plane) and 2 Worker Nodes**, which is an industry-standard architecture used in production environments. This gave me hands-on experience in cluster initialization, node joining, troubleshooting worker node issues, and validating cluster health.

After cluster setup, I deployed an **Nginx pod** and exposed it using **NodePort service**, which helped me understand real-time application deployment, service networking, and how traffic is routed inside Kubernetes.

Finally, I explored the **Master and Worker node components**, which is important to understand how Kubernetes manages scheduling, networking, and container execution.

Importance of This Task

This task is important because it builds strong practical knowledge of:

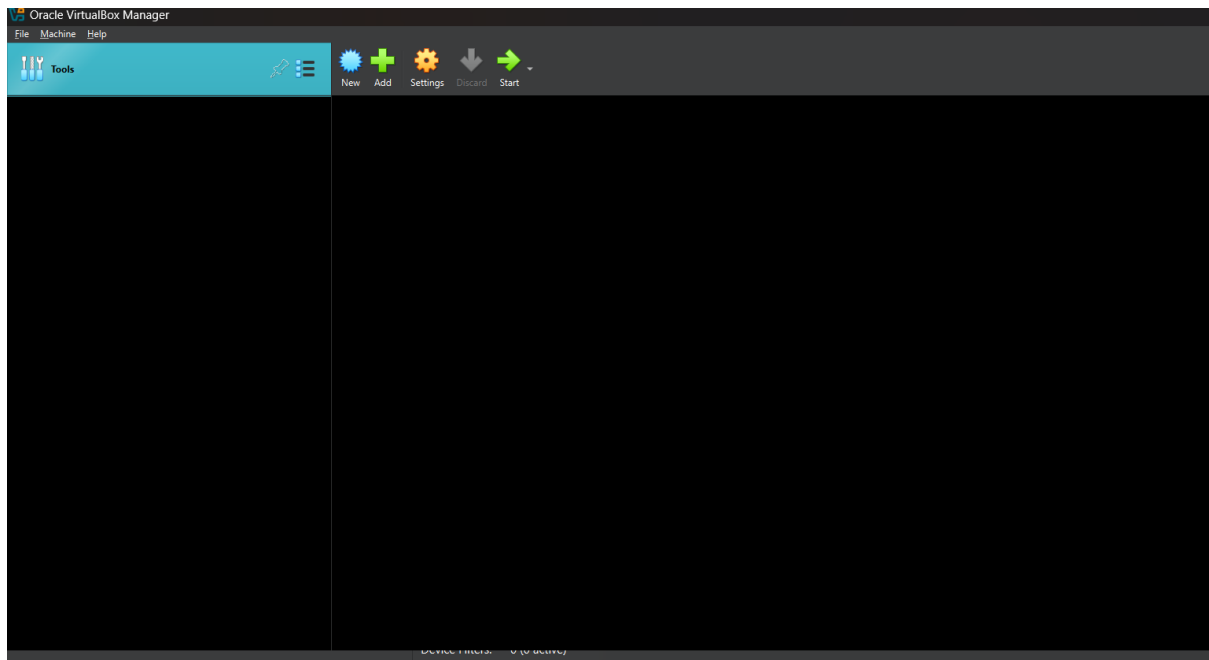
- Kubernetes architecture (Master & Worker responsibilities)
- Cluster setup using kubeadm
- Pod deployment and service exposure
- Troubleshooting real issues (like worker NotReady)
- Real-world DevOps skills used in production deployments

TASK 1: Setup Minikube in Local Machine (Windows)

Step 1: Enable Virtualization

1. Restart PC
2. Enter BIOS (F2 / DEL)
3. Enable **Intel VT-x / AMD-V**

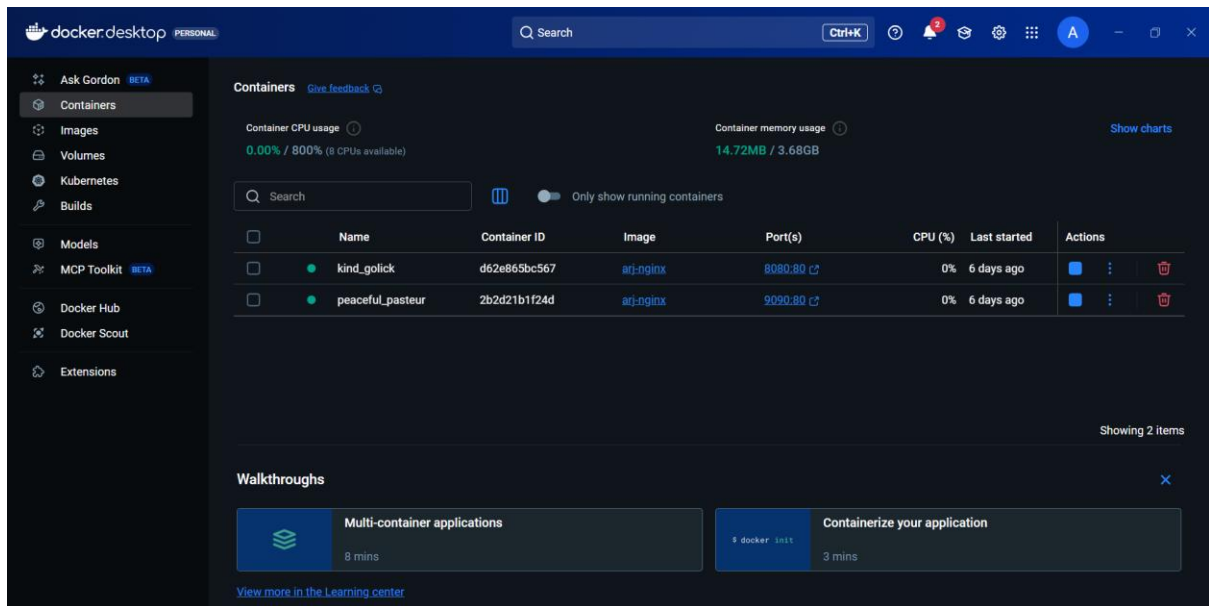
4. Save and Exit



Step 2: Install Docker Desktop

1. Download Docker Desktop from official Docker website
2. Install it
3. Enable these options during installation:
 - ☒ Use WSL2 backend
 - ☒ Add Docker to PATH

Restart PC after installation.



Step 3: Verify Docker Installation

Open PowerShell and run:

docker version

Expected output: Docker Client + Server version.

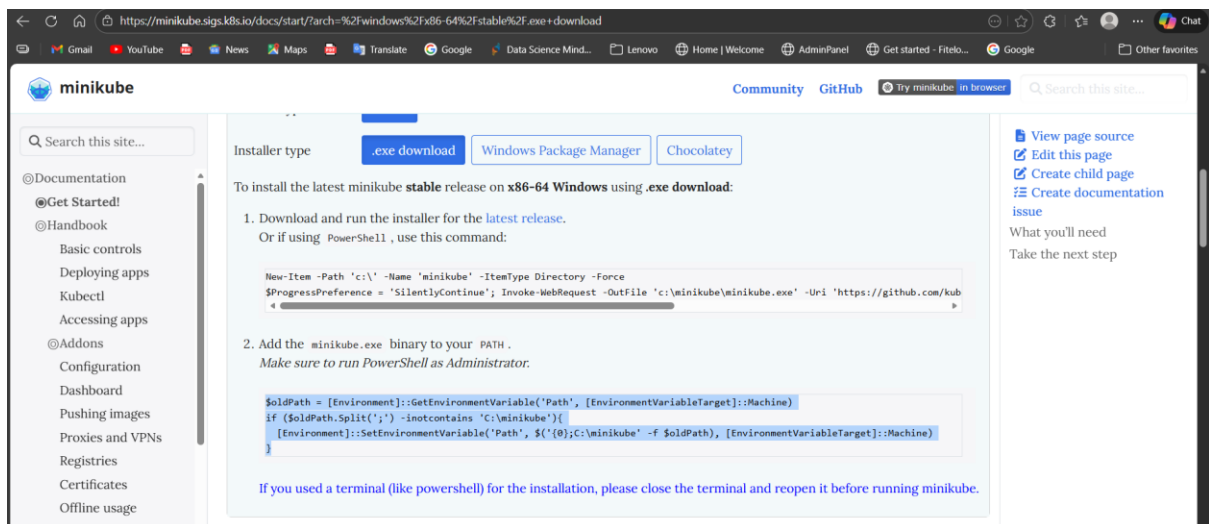
```
>> }
PS C:\WINDOWS\system32> docker --version
Docker version 20.10.1, build e180ab8
PS C:\WINDOWS\system32>
```

Step 4: Install kubectl

Download kubectl:

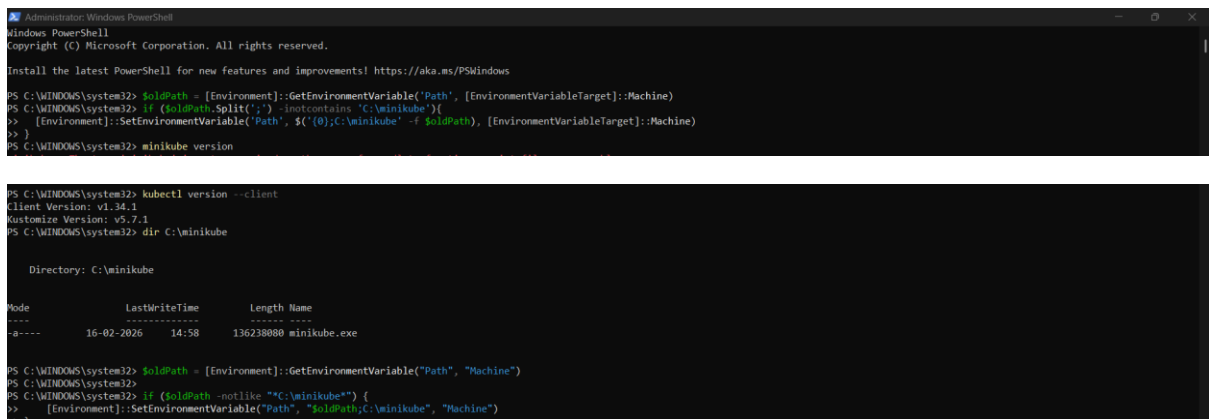
```
curl.exe -LO "https://dl.k8s.io/release/v1.34.1/bin/windows/amd64/kubectl.exe"
```

Create folder and move file:



mkdir C:\kubectl

move kubectl.exe C:\kubectl\



Step 5: Add kubectl to PATH

Go to:

Environment Variables → System Variables → Path → Edit → New

Add:

C:\kubectl

Close and reopen PowerShell.

Verify:

kubectl version --client

Step 6: Install Minikube

Create folder:

```
mkdir C:\minikube
```

Download minikube:

```
curl.exe -LO https://storage.googleapis.com/minikube/releases/latest/minikube-windows-amd64.exe
```

```
move minikube-windows-amd64.exe C:\minikube\minikube.exe
```

Step 7: Add Minikube to PATH

Go to:

Environment Variables → System Variables → Path → Edit → New

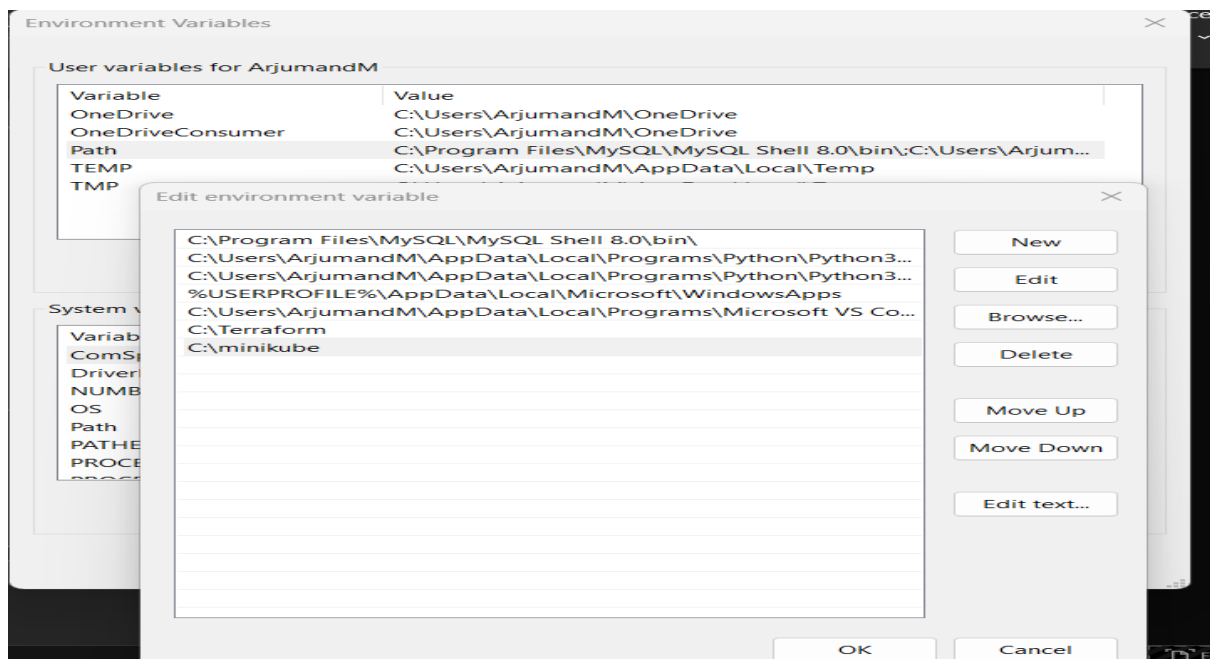
Add:

C:\minikube

Restart PowerShell.

Verify:

minikube version



Step 8: Start Minikube Cluster

Run:

minikube start --driver=docker

```
MINGW64~/c/Users/AgumandM
Arjumand@Arjumand MINGW64 ~ (master)
$ minikube start
* minikube v1.38.0 on Microsoft Windows 11 Home Single Language 24H2
* Automatically selected the docker driver. Other choices: virtualbox, ssh
* Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.49...
```

Step 9: Check Minikube Status

Run:

minikube status

```
Arjumand@Arjumand MINGW64 ~ (master)
$ minikube status
minikube
type: Control Plane
host: Running
kubelt: Running
apiserver: Running
kubeconfig: Configured
```

Step 10: Check Kubernetes Nodes

Run:

kubectl get nodes

Expected output:

minikube Ready

✅ Minikube setup completed.

```
Arjumand@Arjumand MINGW64 ~ (master)
$ kubectl get nodes
NAME     STATUS    ROLES    AGE   VERSION
minikube Ready    control-plane  2m28s  v1.35.0
```

✅ TASK 2: Setup Kubernetes Master and Two Worker Nodes (Ubuntu)

🔥 Requirement

You need **3 Ubuntu Machines (VMs or servers)**:

Node Type Hostname Example IP

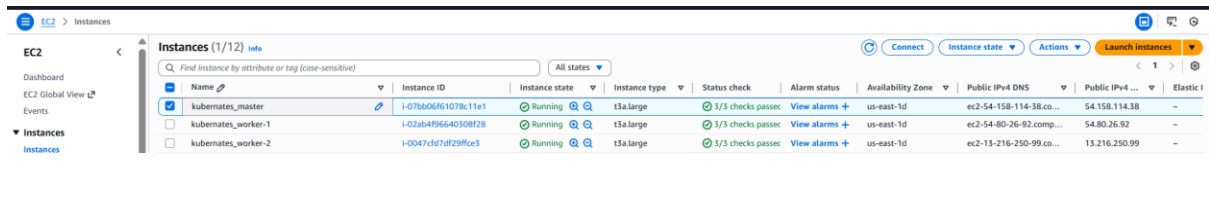
Master master 192.168.1.10

Node Type Hostname Example IP

Worker1 worker1 192.168.1.11

Worker2 worker2 192.168.1.12

All machines must ping each other.



Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic I
kubemates_master	i-07bb06f61078c11e1	Running	t3a.large	3/3 checks passed	View alarms +	us-east-1d	ec2-54-158-114-38.co...	54.158.114.38	-
kubemates_worker-1	i-02ab4f96640308f28	Running	t3a.large	3/3 checks passed	View alarms +	us-east-1d	ec2-54-80-26-92.comp...	54.80.26.92	-
kubemates_worker-2	i-0047c67d729f9ca3	Running	t3a.large	3/3 checks passed	View alarms +	us-east-1d	ec2-13-216-250-99.co...	13.216.250.99	-

◆ Step 1: Set Hostnames (All Nodes)

On Master:

`sudo hostnamectl set-hostname master`

```
ubuntu@master:~$ sudo hostnamectl set-hostname master
root@ip-172-31-32-135:~# hostnamectl set-hostname master
root@ip-172-31-32-135:~# exit
logout
ubuntu@ip-172-31-32-135:~$ exit
logout
Connection to ec2-54-158-114-38.compute-1.amazonaws.com closed.
ubuntu@master:~$ ssh -i "mykey" ubuntu@ec2-54-158-114-38.compute-1.amazonaws.com
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Feb 16 10:29:49 UTC 2026

System load:  0.0      Temperature:   -273.1 C
Usage of /:   6.3% of 28.0GB    Processes:    111
Memory usage: 2%      Users logged in: 0
Swap usage:  0%      IPv4 address for ens5: 172.31.32.135

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Mon Feb 16 10:21:32 2026 from 14.192.14.60
ubuntu@master:~$
```

On Worker1:

`sudo hostnamectl set-hostname worker1`



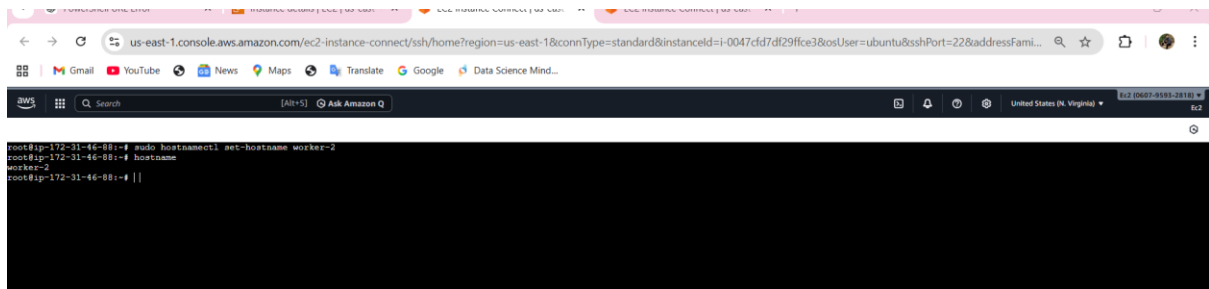
```
root@worker-1:~# sudo hostnamectl set-hostname worker1
root@worker-1:~# hostnamectl set-hostname worker1
root@worker-1:~#
```

On Worker2:

`sudo hostnamectl set-hostname worker2`

Restart all nodes:

`sudo reboot`

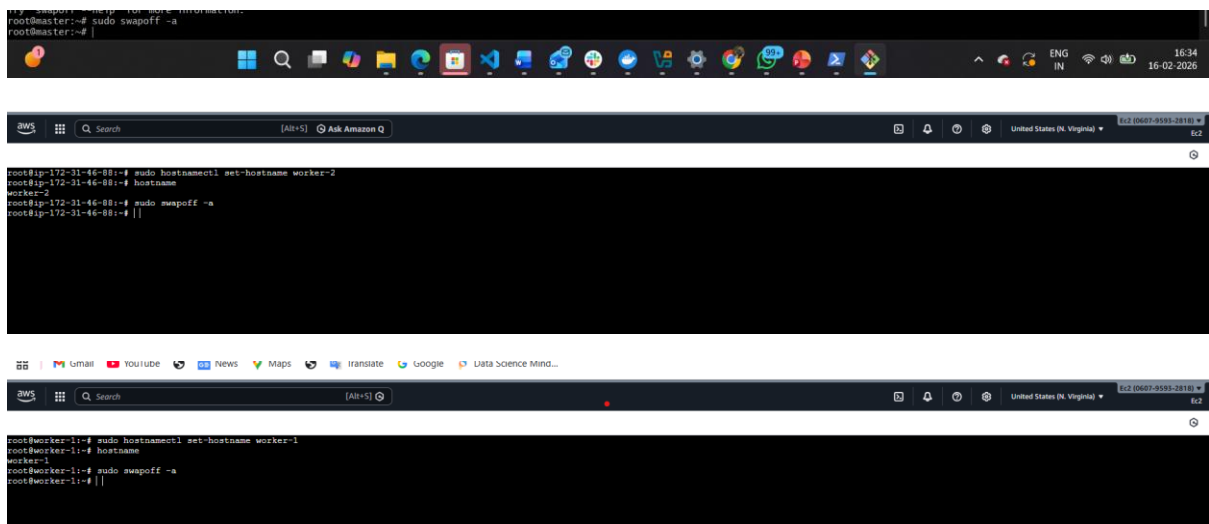


```
root@ip-172-31-46-88:~# sudo hostnamectl set-hostname worker-2
root@ip-172-31-46-88:~# hostname
worker-2
root@ip-172-31-46-88:~# ||
```

Do The below from steps 1 to 6 in both Master & Worker nodes:-

Step 1: Disable swap:

`sudo swapoff -a`



```
root@master:~# sudo swapoff -a
root@master:~# |

root@ip-172-31-46-88:~# sudo hostnamectl set-hostname worker-2
root@ip-172-31-46-88:~# hostname
worker-2
root@ip-172-31-46-88:~# sudo swapoff -a
root@ip-172-31-46-88:~# ||

root@worker-1:~# sudo hostnamectl set-hostname worker-1
root@worker-1:~# hostname
worker-1
root@worker-1:~# sudo swapoff -a
root@worker-1:~# ||
```

Step 3: Update the /etc/hosts File for Hostname Resolution

`sudo vi /etc/hosts`

Add private ip's of nodes



```
root@master:~#
172.31.32.135 master
172.31.46.47 worker-1
172.31.46.88 worker-2
```

Example:-

Step 4: Set up the IPV4 bridge on all nodes

`cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf`

`overlay`

`br_netfilter`

`EOF`


```

root@master:~# sudo vi /etc/hosts
root@master:~# cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
root@master:~#

```



sudo modprobe overlay

sudo modprobe br_netfilter

sysctl params required by setup, params persist across reboots

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf

net.bridge.bridge-nf-call-iptables = 1

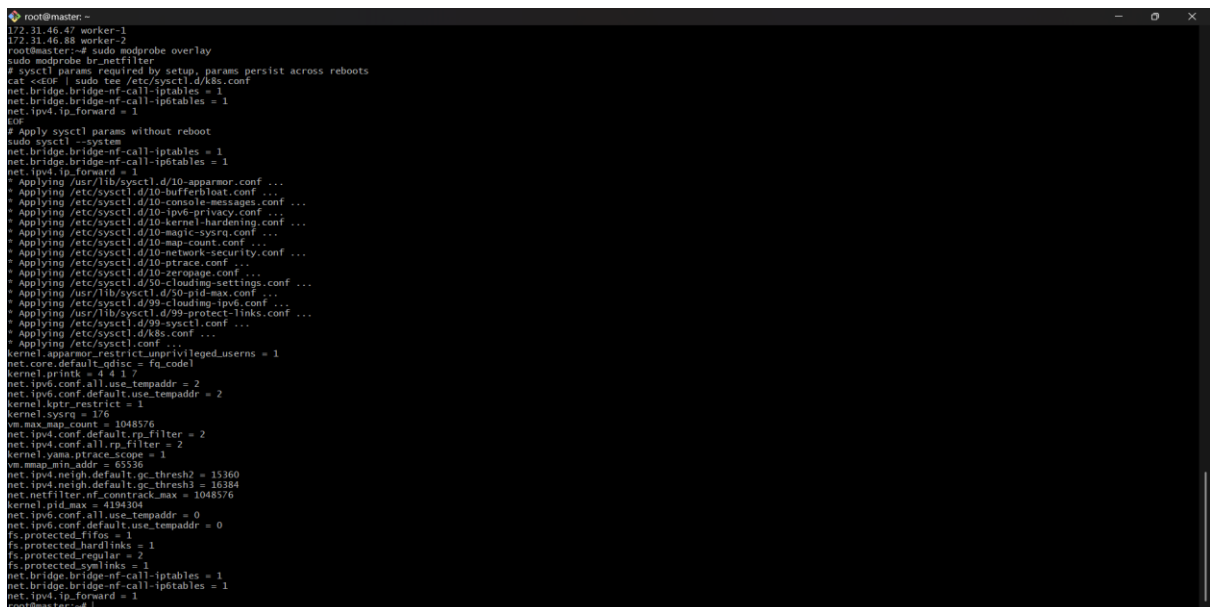
net.bridge.bridge-nf-call-ip6tables = 1

net.ipv4.ip_forward = 1

EOF

Apply sysctl params without reboot

sudo sysctl -system



→worker-1

```
* Applying /etc/sysctl.d/10-console-messages.conf ...
* Applying /etc/sysctl.d/10-ipw6-privacy.conf ...
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-network-security.conf ...
* Applying /etc/sysctl.d/10-ptrace.conf ...
* Applying /etc/sysctl.d/10-utmpd.conf ...
* Applying /etc/sysctl.d/50-cloudimg-settings.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /etc/sysctl.d/99-protect-links.conf ...
* Applying /usr/lib/sysctl.d/99-sysctl.conf ...
* Applying /etc/sysctl.d/99.conf ...
* Applying /etc/sysctl.d/99.conf ...
kernel.apparmor_restrict_unprivileged_usersns = 1
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
kernel.kptr_restrict = 1
kernel.sysrq = 176
vm.max_map_count = 1048576
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
kernel.yama.ptrace_scope = 1
vm.mmap_min_addr = 65536
net.ipv4.neigh.default.gc_thresh2 = 15360
net.ipv4.neigh.default.gc_thresh3 = 16384
net.netfilter.nf_conntrack_max = 1048576
kernel.pid_max = 4194304
net.ipv6.conf.all.use_tempaddr = 0
net.ipv6.conf.default.use_tempaddr = 0
fs.protected_fifos = 1
fs.protected_hardlinks = 1
fs.protected_regular = 2
fs.protected_symlinks = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
root@worker-1:~#

i-02ab4f96640308f28 (kubernetes_worker-1)
PublicIP: 54.80.26.92 PrivateIP: 172.31.46.47
```

→Worker-2

```
* Applying /etc/sysctl.d/10-console-messages.conf ...
* Applying /etc/sysctl.d/10-ipw6-privacy.conf ...
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-network-security.conf ...
* Applying /etc/sysctl.d/10-ptrace.conf ...
* Applying /etc/sysctl.d/10-utmpd.conf ...
* Applying /etc/sysctl.d/50-cloudimg-settings.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /etc/sysctl.d/99-protect-links.conf ...
* Applying /usr/lib/sysctl.d/99-sysctl.conf ...
* Applying /etc/sysctl.d/99.conf ...
* Applying /etc/sysctl.d/99.conf ...
kernel.apparmor_restrict_unprivileged_usersns = 1
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
kernel.kptr_restrict = 1
kernel.sysrq = 176
vm.max_map_count = 1048576
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
kernel.yama.ptrace_scope = 1
vm.mmap_min_addr = 65536
net.ipv4.neigh.default.gc_thresh2 = 15360
net.ipv4.neigh.default.gc_thresh3 = 16384
net.netfilter.nf_conntrack_max = 1048576
kernel.pid_max = 4194304
net.ipv6.conf.all.use_tempaddr = 0
net.ipv6.conf.default.use_tempaddr = 0
fs.protected_fifos = 1
fs.protected_hardlinks = 1
fs.protected_regular = 2
fs.protected_symlinks = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
root@worker-2:~#

i-0047cfd7df29f3ce3 (kubernetes_worker-2)
PublicIP: 13.216.250.99 PrivateIP: 172.31.46.88
```

Step 5: Install kubelet, kubeadm, and kubectl on each node

sudo apt-get update

sudo apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]

https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | sudo tee

/etc/apt/sources.list.d/kubernetes.list

sudo apt-get update

sudo apt-get install -y kubelet kubeadm kubectl

sudo apt-mark hold kubelet kubeadm kubectl

→ Master–

```

cri-tools kubernetescni
cri-tools kubeadm kubectl kubenet kubernetescni
# upgraded, 3 newly installed, 0 to remove and 73 not upgraded.
Need to get 93.9 MB of archives.
After this operation, 333 MB of additional disk space will be used.
Get: https://prod-cdn.packages.k8s.io/repositorie/txv/kubernetescni/core/stable/v1.34.0-1.1 [16.7 MB]
Get: https://prod-cdn.packages.k8s.io/repositorie/txv/kubernetescni/core/stable/v1.34.0-1.1 [12.9 MB]
Get: https://prod-cdn.packages.k8s.io/repositorie/txv/kubernetescni/core/stable/v1.34.0-1.1 [11.7 MB]
Get: https://prod-cdn.packages.k8s.io/repositorie/txv/kubernetescni/core/stable/v1.34.0-1.1 [39.9 MB]
Get: https://prod-cdn.packages.k8s.io/repositorie/txv/kubernetescni/core/stable/v1.34.0-1.1 [13.0 MB]
Fetched 93.9 MB in 1s (68.2 MB/s)
Reading previously unslected package cri-tools.
(Reading database ... 72322 files and directories currently installed.)
Preparing to unpack .../cri-tools_1.34.0-1.1_amd64.deb ...
Unpacking cri-tools (1.34.0-1.1) ...
Selecting previously unslected package kubeadm.
Preparing to unpack .../kubeadm_1.34.0-1.1_amd64.deb ...
Unpacking kubeadm (1.34.0-1.1) ...
Selecting previously unslected package kubectl.
Preparing to unpack .../kubectl_1.34.0-1.1_amd64.deb ...
Unpacking kubectl (1.34.0-1.1) ...
Selecting previously unslected package kubernetescni.
Preparing to unpack .../kubernetescni_1.34.0-1.1_amd64.deb ...
Unpacking kubernetescni (1.34.0-1.1) ...
Preparing to unpack .../kubenet_1.34.0-1.1_amd64.deb ...
Unpacking kubenet (1.34.0-1.1) ...
Setting up cri-tools (1.34.0-1.1) ...
Setting up kubernetescni (1.34.0-1.1) ...
Setting up kubeadm (1.34.0-1.1) ...
Setting up kubectl (1.34.0-1.1) ...
Setting up kubenet (1.34.0-1.1) ...
Cleaning processes...
Scanning Linux images...
Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@master:~# sudo apt-mark hold kubenet kubeadm kubectl
kubenet set on hold.
kubeadm set on hold.
kubectl set on hold.
kubernetescni set on hold.
kubenet version --client
kubeadm version --server
Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GtVersion:"v1.34.4", GtCommit:"14507e2fb3b1d7272ccba80bc282c2
Client Version: v1.34.4
K8sVersion: v5.7.1
kubernetescni v1.34.4
root@master:~#

```

→ Worker-1

```
Unpacking kubelet (1.34.4-1.1) ...
Setting up kubectl (1.34.4-1.1) ...
Setting up cri-tools (1.34.0-1.1) ...
Setting up kubernetes-cni (1.7.1-1.1) ...
Setting up kubeadm (1.34.4-1.1) ...
Setting up kubelet (1.34.4-1.1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@worker-1:~# sudo apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
root@worker-1:~# kubeadm version
kubelet version --client
kubelet --version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.4", GitCommit:"14507e2fb33b1d2712c2c8baed0bc282c27a4a04a", GitTreeState:"clean", BuildDate:"2026-02-10T12:55:46Z", GoVersion:"go1.24.12", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.4
kubelet version: v5.7.1
kubernetes v1.34.4
root@worker-1:~# []
```

→ Worker-2

```
Unpacking kubectl (1.34.4-1.1) ...
Setting up kubectl (1.34.4-1.1) ...
Setting up cri-tools (1.34.0-1.1) ...
Setting up kubernetes-cni (1.7.1-1.1) ...
Setting up kubeadm (1.34.4-1.1) ...
Setting up kubelet (1.34.4-1.1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@worker-2:~# sudo apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
root@worker-2:~# kubeadm version
kubelet version --client
kubelet --version
kubeadm version: version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.4", GitCommit:"14507e2fb1b1d2712c3baed0bc282c274a04a", GitTreeState:"clean", BuildDate:"2026-02-10T12:55:46Z", GoVersion:"go1.24.12", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.4
Customize Version: v5.7.1
kubernetes v1.34.4
root@worker-2:~#
```

Step 6: Install Docker

```
sudo apt install docker.io
```

```
sudo mkdir /etc/containerd
```

sudo sh -c "containerd config default > /etc/containerd/config.toml"

sudo sed -i 's/ SystemdCgroup = false/ SystemdCgroup = true/' /etc/containerd/config.toml

sudo systemctl restart containerd.service

sudo systemctl restart kubelet.service

sudo systemctl enable kubelet.service

→master

```
kubernetes v1.34.4
root@master:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-16 10:54:18 UTC; 1h 41min ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 2259 (dockerd)
      Tasks: 9
     Memory: 21.1M (peak: 22.0M)
        CPU: 1.342s
    CGroup: /system.slice/docker.service
            └─2259 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Feb 16 10:54:18 master dockerd[2259]: time="2026-02-16T10:54:18.244398859Z" level=info msg="Loading containers: start."
Feb 16 10:54:18 master dockerd[2259]: time="2026-02-16T10:54:18.728512792Z" level=info msg="Loading containers: done."
Feb 16 10:54:18 master dockerd[2259]: time="2026-02-16T10:54:18.762370482Z" level=info msg="Docker daemon" commit="28.2.2-0ubuntu1-24.04.1" containerd-snapshotter=false storage-driver=overlay2 version=28.2.2
Feb 16 10:54:18 master dockerd[2259]: time="2026-02-16T10:54:18.762707399Z" level=info msg="Initializing buildkit"
Feb 16 10:54:18 master dockerd[2259]: time="2026-02-16T10:54:18.773070702Z" level=warning msg="CDI setup error /var/run/cdi: failed to monitor for changes: no such file or directory"
Feb 16 10:54:18 master dockerd[2259]: time="2026-02-16T10:54:18.773109332Z" level=warning msg="CDI setup error /etc/cdi: failed to monitor for changes: no such file or directory"
Feb 16 10:54:18 master dockerd[2259]: time="2026-02-16T10:54:18.795351398Z" level=info msg="Completed buildkit initialization"
Feb 16 10:54:18 master dockerd[2259]: time="2026-02-16T10:54:18.804860536Z" level=info msg="Daemon has completed initialization"
Feb 16 10:54:18 master dockerd[2259]: time="2026-02-16T10:54:18.805117667Z" level=info msg="API listen on /run/docker.sock"
Feb 16 10:54:18 master systemd[1]: Started docker.service - Docker Application Container Engine.
root@master:~#
```

→worker-1

```
kubeadm version: kversion.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.4", GitCommit:"14507e2fb33b1d2712ccbaed0bc282c274a04a", GitTreeState:"clean", BuildDate:"2026-02-10T12:55:46Z", GoVersion:"go1.24.12", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.4
Kustomize Version: v5.7.1
Kubernetes v1.34.4
root@worker-1:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-16 11:19:43 UTC; 1h 17min ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 2858 (dockerd)
      Tasks: 9
     Memory: 21.0M (peak: 22.0M)
        CPU: 1.178s
    CGroup: /system.slice/docker.service
            └─2858 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Feb 16 11:19:43 worker-1 dockerd[2858]: time="2026-02-16T11:19:43.166752218Z" level=info msg="Loading containers: start."
Feb 16 11:19:43 worker-1 dockerd[2858]: time="2026-02-16T11:19:43.633104798Z" level=info msg="Loading containers: done."
Feb 16 11:19:43 worker-1 dockerd[2858]: time="2026-02-16T11:19:43.664171495Z" level=info msg="Docker daemon" commit="28.2.2-0ubuntu1-24.04.1" containerd-snapshotter=false storage-driver=
Feb 16 11:19:43 worker-1 dockerd[2858]: time="2026-02-16T11:19:43.664341878Z" level=info msg="Initializing buildkit"
Feb 16 11:19:43 worker-1 dockerd[2858]: time="2026-02-16T11:19:43.675020846Z" level=warning msg="CDI setup error /etc/cdi: failed to monitor for changes: no such file or directory"
Feb 16 11:19:43 worker-1 dockerd[2858]: time="2026-02-16T11:19:43.675069187Z" level=warning msg="CDI setup error /var/run/cdi: failed to monitor for changes: no such file or directory"
Feb 16 11:19:43 worker-1 dockerd[2858]: time="2026-02-16T11:19:43.697103704Z" level=info msg="Completed buildkit initialization"
Feb 16 11:19:43 worker-1 dockerd[2858]: time="2026-02-16T11:19:43.709674043Z" level=info msg="Daemon has completed initialization"
Feb 16 11:19:43 worker-1 systemd[1]: Started docker.service - Docker Application Container Engine.
Feb 16 11:19:43 worker-1 dockerd[2858]: time="2026-02-16T11:19:43.712155424Z" level=info msg="API listen on /run/docker.sock"
lines 1-22/22 [END]
```

i-02ab4f96640308f28 (kubernetes_worker-1)
PublicIPs: 54.80.26.92 PrivateIPs: 172.31.46.47

→worker-2

```
kubectl version --client
kubelet --version
kubeadm version: kversion.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.4", GitCommit:"14507e2fb33b1d2712ccbaed0bc282c274a04a", GitTreeState:"clean", BuildDate:"2026-02-10T12:55:46Z", GoVersion:"go1.24.12", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.4
Kustomize Version: v5.7.1
Kubernetes v1.34.4
root@worker-2:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-16 11:19:49 UTC; 1h 17min ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 2542 (dockerd)
      Tasks: 9
     Memory: 21.0M (peak: 21.9M)
        CPU: 1.371s
    CGroup: /system.slice/docker.service
            └─2542 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Feb 16 11:19:49 worker-2 dockerd[2542]: time="2026-02-16T11:19:49.107664494Z" level=info msg="Loading containers: start."
Feb 16 11:19:49 worker-2 dockerd[2542]: time="2026-02-16T11:19:49.805983785Z" level=info msg="Loading containers: done."
Feb 16 11:19:49 worker-2 dockerd[2542]: time="2026-02-16T11:19:49.849855973Z" level=info msg="Docker daemon" commit="28.2.2-0ubuntu1-24.04.1" containerd-snapshotter=false storage-driver=
Feb 16 11:19:49 worker-2 dockerd[2542]: time="2026-02-16T11:19:49.850437130Z" level=info msg="Initializing buildkit"
Feb 16 11:19:49 worker-2 dockerd[2542]: time="2026-02-16T11:19:49.862714018Z" level=warning msg="CDI setup error /etc/cdi: failed to monitor for changes: no such file or directory"
Feb 16 11:19:49 worker-2 dockerd[2542]: time="2026-02-16T11:19:49.862774319Z" level=warning msg="CDI setup error /var/run/cdi: failed to monitor for changes: no such file or directory"
Feb 16 11:19:49 worker-2 dockerd[2542]: time="2026-02-16T11:19:49.888387978Z" level=info msg="Completed buildkit initialization"
Feb 16 11:19:49 worker-2 dockerd[2542]: time="2026-02-16T11:19:49.903004776Z" level=info msg="Daemon has completed initialization"
Feb 16 11:19:49 worker-2 dockerd[2542]: time="2026-02-16T11:19:49.903151965Z" level=info msg="API listen on /run/docker.sock"
Feb 16 11:19:49 worker-2 systemd[1]: Started docker.service - Docker Application Container Engine.
lines 1-22/22 [END]
```

i-0047cfd7df29f3e3 (kubernetes_worker-2)
PublicIPs: 13.216.250.99 PrivateIPs: 172.31.46.88

◆ Step 3: Execute below commands

docker --version

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@master:~# docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

systemctl start docker

#systemctl enable docker

systemctl status docker

```
root@master:~# systemctl start docker
root@master:~# systemctl enable docker
root@master:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-16 10:54:18 UTC; 54s ago
     Triggers: ● docker.socket
   Docs: https://docs.docker.com
   Main PID: 2259 (dockerd)
    Tasks: 9
   Memory: 21.0M (peak: 22.0M)
      CPU: 551ms
   CGroup: /system.slice/docker.service
           └─2259 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

sudo mkdir /etc/containerd

sudo sh -c "containerd config default > /etc/containerd/config.toml"

sudo sed -i 's/ SystemdCgroup = false/ SystemdCgroup = true/' /etc/containerd/config.toml

sudo systemctl restart containerd.service

sudo systemctl restart kubelet.service

sudo systemctl enable kubelet.service

→master

```
root@master:~# sudo mkdir /etc/containerd
root@master:~# sudo sh -c "containerd config default > /etc/containerd/config.toml"
root@master:~# sudo sed -i 's/ SystemdCgroup = false/ SystemdCgroup = true/' /etc/containerd/config.toml
root@master:~# sudo systemctl restart containerd.service
root@master:~# sudo systemctl enable kubelet.service
root@master:~# sudo systemctl status containerd --no-pager
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-16 12:41:09 UTC; 1min 44s ago
     Docs: https://containerd.io
   Main PID: 4969 (containerd)
    Tasks: 8
   Memory: 13.4M (peak: 13.8M)
      CPU: 308ms
   CGroup: /system.slice/containerd.service
           └─4969 /usr/bin/containerd

Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.583911522Z" level=info msg=serving... address=/run/containerd/containerd.sock.ttrpc
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.583998812Z" level=info msg=serving... address=/run/containerd/containerd.sock
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584133814Z" level=info msg="Start subscribing containerd event"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584351746Z" level=info msg="Start recovering state"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584732882Z" level=info msg="Start event monitor"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584508388Z" level=info msg="Start snapshots syncer"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584525922Z" level=info msg="Start cni network conf syncer for default"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.584506682Z" level=info msg="Start streaming server"
Feb 16 12:41:09 master containerd[4969]: time="2026-02-16T12:41:09.585278438Z" level=info msg="containerd successfully booted in 0.057939s"
root@master:~# sudo systemctl status kubelet --no-pager
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/usr/lib/systemd/system/kubelet.service; enabled; preset: enabled)
   Drop-In: /usr/lib/systemd/system/kubelet.service.d
           └─10-kubeadm.conf
   Active: activating (auto-restart) (Result: exit-code) since Mon 2026-02-16 12:43:02 UTC; 4s ago
     Docs: https://kubernetes.io/docs/
   Process: 5129 ExecStart=/usr/bin/kubelet KUBELET_KUBECONFIG_ARGS KUBELET_CONFIG_ARGS KUBELET_KUBEADM_ARGS KUBELET_EXTRA_ARGS (code=exited, status=1/FAILURE)
   Main PID: 5129 (code=exited, status=1/FAILURE)
      CPU: 8ms
root@master:~#
```

→Worker-1

```
Active: active (running) since Mon 2026-02-16 12:43:50 UTC; 15s ago
     Docs: https://containerd.io
   Main PID: 5272 (containerd)
    Tasks: 7
   Memory: 13.7M (peak: 14.0M)
      CPU: 137ms
   CGroup: /system.slice/containerd.service
           └─5272 /usr/bin/containerd

Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027046645Z" level=error msg="failed to load cni during init, please check CRI plugin status before set ad cni config"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027248678Z" level=info msg="Start subscribing containerd event"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027313870Z" level=info msg="Start recovering state"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027397121Z" level=info msg="Start event monitor"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027424541Z" level=info msg="Start snapshots syncer"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027442291Z" level=info msg="Start cni network conf syncer for default"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027456402Z" level=info msg="Start streaming server"
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027480502Z" level=info msg=serving... address=/run/containerd/containerd.sock.ttrpc
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027555073Z" level=info msg=serving... address=/run/containerd/containerd.sock
Feb 16 12:43:50 worker-1 containerd[5272]: time="2026-02-16T12:43:50.027653855Z" level=info msg="containerd successfully booted in 0.049327s"
Hint: Some lines were ellipsized, use -l to show in full.
root@worker-1:~# sudo systemctl status kubelet --no-pager
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/usr/lib/systemd/system/kubelet.service; enabled; preset: enabled)
   Drop-In: /usr/lib/systemd/system/kubelet.service.d
           └─10-kubeadm.conf
   Active: activating (auto-restart) (Result: exit-code) since Mon 2026-02-16 12:44:11 UTC; 9s ago
     Docs: https://kubernetes.io/docs/
   Process: 5153 ExecStart=/usr/bin/kubelet KUBELET_KUBECONFIG_ARGS KUBELET_CONFIG_ARGS KUBELET_KUBEADM_ARGS KUBELET_EXTRA_ARGS (code=exited, status=1/FAILURE)
   Main PID: 5153 (code=exited, status=1/FAILURE)
      CPU: 8ms
root@worker-1:~#
```

i-02ab4f96640308f28 (kubernetes_worker-1)
PublicIPs: 54.80.26.92 PrivateIPs: 172.31.46.47

→worker-2

```
root@worker-2:~# sudo mkdir /etc/containerd
sudo sh -c "containerd config default > /etc/containerd/config.toml"
sudo sed -i 's/ SystemdCgroup = false/ SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd.service
sudo systemctl restart kubelet.service
sudo systemctl enable kubelet.service
root@worker-2:~# sudo systemctl status containerd --no-pager
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-16 12:43:56 UTC; 17s ago
     Docs: https://containerd.io
   Main PID: 5326 (containerd)
      Tasks: 7
   Memory: 13.0M (peak: 13.8M)
        CPU: 165ms
   CGroup: /system.slice/containerd.service
           └─5326 /usr/bin/containerd

Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.752468596Z" level=error msg="failed to load cni during init, please check CRI plugin status before set ad cni config"
Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.752703839Z" level=info msg="Start subscribing containerd event"
Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.752788330Z" level=info msg="Start recovering state"
Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.752883581Z" level=info msg="Start event monitor"
Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.752887521Z" level=info msg="serving... address=/run/containerd/containerd.sock.ttrpc"
Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.752924991Z" level=info msg="Start snapshots syncer"
Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.753002272Z" level=info msg="serving... address=/run/containerd/containerd.sock"
Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.753020492Z" level=info msg="Start cni network conf syncer for default"
Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.753125074Z" level=info msg="Start streaming server"
Feb 16 12:43:56 worker-2 containerd[5326]: time="2026-02-16T12:43:56.753209585Z" level=info msg="containerd successfully booted in 0.050346s"
Hint: Some lines were ellipsized; use -l to show in full.
root@worker-2:~# sudo systemctl status kubelet --no-pager
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/usr/lib/systemd/system/kubelet.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-16 12:43:56 UTC; 17s ago
     Docs: https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/
   Main PID: 5326 (kubelet)
      Tasks: 7
   Memory: 13.0M (peak: 13.8M)
        CPU: 165ms
   CGroup: /system.slice/kubelet.service
           └─5326 /usr/bin/kubelet
```

i-0047cfd7df29ffce3 (kubernetes_worker-2)

PublicIPs: 13.216.250.99 PrivateIPs: 172.31.46.88

Step 7: Initialize the Kubernetes cluster on the master node

On MASTER NODE:-

sudo kubeadm config images pull

sudo kubeadm init --pod-network-cidr=10.10.0.0/16

mkdir -p \$HOME/.kube

sudo cp -i /etc/kubernetes/admin.conf \$HOME/.kube/config

sudo chown \$(id -u):\$(id -g) \$HOME/.kube/config

```
[wait-control-plane] waiting for the kubelet to boot up the control plane as static Pods from directory "/etc/kubernetes/manifests"
[kubelet-check] waiting for a healthy kubelet at https://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.50157367s
[control-plane-check] waiting for healthy control plane components. This can take up to 4m0s
[control-plane-check] Checking kube-apiserver at https://127.31.32.135:6443/liveness
[control-plane-check] Checking kube-controller-manager at https://127.0.0.1:10257/healthz
[control-plane-check] Checking kube-scheduler at https://127.0.0.1:10259/liveness
[control-plane-check] kube-controller-manager is healthy after 4.175034336s
[control-plane-check] kube-scheduler is healthy after 5.627630783s
[control-plane-check] kube-apiserver is healthy after 7.501661695s
[upload-config] Storing the configuration used in ConfigMap "kubeadm-config" in the "kube-system" Namespace
[kubelet] Creating a ConfigMap "kubelet-config" in namespace kube-system with the configuration for the kubelets in the cluster
[upload-certs] Skipping phase. Please see --upload-certs
[mark-control-plane] Marking the node master as control-plane by adding the labels [node-role.kubernetes.io/control-plane:node.kubernetes.io/exclude-from-external-load-balancers]
[bootstrap-token] Using token: 809ec2.rzftm0ts6w7j75
[bootstrap-token] Configuring bootstrap tokens, cluster-info ConfigMap, RBAC Roles
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to get nodes
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to post CSRs in order for nodes to get long term certificate credentials
[bootstrap-token] Configured RBAC rules to allow the csrapprover controller automatically approve CSRs from a Node Bootstrap Token
[bootstrap-token] Configured RBAC rules to allow certificate rotation for all node client certificates in the cluster
[bootstrap-token] Creating the "cluster-info" ConfigMap in the "kube-public" namespace
[kubelet-finalize] updating "/etc/kubernetes/kubelet.conf" to point to a rotatable kubelet client certificate and key
[addons] Applied essential addons: CoreDNS
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 172.31.32.135:6443 --token 809ec2.rzftm0ts6w7j75 \
--discovery-token-ca-cert-hash sha256:64b52247e97cd2cc50773fcf4901b66406705865501c9047609a28c1da01df8

root@master:~# mkdir -p $HOME/.kube
root@master:~# sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
root@master:~# sudo chown $(id -u):$(id -g) $HOME/.kube/config
Step 8: Configure kubectl and Calico
Command 'step' not found, did you mean:
  Command 'step' from deb step (4:23.08.4-0ubuntu1)
Try: apt install sdeb name
root@master:~# mkdir -p $HOME/.kube
root@master:~# sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
root@master:~# sudo chown $(id -u):$(id -g) $HOME/.kube/config
cat -overtite /root/.kube/config? yes
```

Step 8: Configure kubectl and Calico

kubectl create -f

<https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/tigera-operator.yaml>


```
curl https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/custom-resources.yaml -O
```

```
sed -i 's/cidr: 192\168\0\0\16/cidr: 10.10.0.0\16/g' custom-resources.yaml
```

```
kubectl create -f custom-resources.yaml
```

```
root@master:~# kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/tigera-operator.yaml
root@master:~# curl https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/custom-resources.yaml -O
root@master:~# sed -i 's/cidr: 192\168\0\0\16/cidr: 10.10.0.0\16/g' custom-resources.yaml
root@master:~# kubectl create -f custom-resources.yaml
namespace/tigera-operator created
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/blockaffinities.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/caliconodestatuses.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/clusterinformations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/felixconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/globalnetworkpolicies.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/globalnetworksets.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/hostendpoints.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamblocks.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamconfigs.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamhandles.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipnps.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipreservations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/kubecontrollersconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/networkpolicies.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/networksets.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/apiservers.operator.tigera.io created
customresourcedefinition.apiextensions.k8s.io/imagessets.operator.tigera.io created
customresourcedefinition.apiextensions.k8s.io/installations.operator.tigera.io created
customresourcedefinition.apiextensions.k8s.io/tigerastatuses.operator.tigera.io created
serviceaccount/tigera-operator created
clusterrole.rbac.authorization.k8s.io/tigera-operator created
clusterrolebinding.rbac.authorization.k8s.io/tigera-operator created
deployment.apps/tigera-operator created
100 824 100 824 0 0 6651 0 --:--:-- --:--:-- --:--:-- 6699
installation.operator.tigera.io/default created
apiserver.operator.tigera.io/default created
root@master:~#
```

Step 9: Add worker nodes to the cluster

```
sudo kubeadm join <MASTER_NODE_IP>:<API_SERVER_PORT> --token <TOKEN> --discovery-token-ca-cert-hash
```

```
<CERTIFICATE_HASH>
```

Step 10: Verify the cluster and test

```
kubectl get nodes
```

```
root@master:~# kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
master        Ready     control-plane   119m   v1.34.4
worker-1      Ready     <none>        3m25s   v1.34.4
worker-2      Ready     <none>        3m22s   v1.34.4
```

TASK 3: Run One Nginx Pod in Kubernetes Cluster

◆ Step 1: Create Nginx Pod (MASTER)

Run:

```
kubectl run nginx --image=nginx --port=80
```

```
root@master:~# kubectl run nginx --image=nginx --port=80
pod/nginx created
root@master:~#
```

◆ Step 2: Check Pod Status

nginx Running

```
root@master:~# kubectl run nginx --image=nginx --port=80
pod/nginx created
root@master:~# kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           29s
root@master:~#
```

◆ Step 3: Describe Pod (Optional)

kubectl describe pod nginx

```

NAME: READY STATUS RESTARTS AGE
nginx 1/1 Running 0 29s
root@master:~# kubectl describe pod nginx
Name:
Namespace: default
Priority: 0
Service Account: default
Node: worker-2/172.31.46.88
Start Time: Mon, 16 Feb 2026 14:50:59 +0000
Labels:
  run/nginx:
  cnl-projectcalico.org/containerID: 0e1616ac6c652ee8674db9b4da7ac21e0b332575df61d6d386d30ca176b74022
  cnl-projectcalico.org/podIP: 10.10.133.194/32
  cnl-projectcalico.org/podIPs: 10.10.133.194/32
Annotations:
Status: Running
IP: 10.10.133.194
IPs:
  IP: 10.10.133.194
Containers:
  nginx:
    Container ID: containerd://d195072c4a5a1b21001c5fef1843c2de6b442520f6cf4eac4494fc561af39da3
    Image: nginx
    Image ID: docker.io/library/nginx@sha256:341bf03f3c6c5277d6002cf6e1fb0319fa4252ad24ab8a0e262e0059d313208
    Port: 80/TCP
    Host Port: 0/TCP
    State: Running
      Started: Mon, 16 Feb 2026 14:51:04 +0000
    Ready: True
    Restart Count: 0
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-mgplw (ro)
Conditions:
  Type Status
  PodReadyToStartContainers True
  Initialized True
  Ready True
  ContainersReady True
  PodScheduled True
Volumes:
  kube-api-access-mgplw:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
  qos-class:
    BestEffort
Node-Selectors:
  <none>
Tolerations:
  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
  node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type Reason Age From Message
  ----
  Normal Scheduled 54s default-scheduler Successfully assigned default/nginx to worker-2
  Normal Pulling 53s kubelet Pulling image "nginx"
  Normal Pulled 49s kubelet Successfully pulled image "nginx" in 3.699s (3.699s including waiting). Image size: 62939286 bytes.
  Normal Created 49s kubelet Created container nginx
  Normal Started 49s kubelet Started container nginx

```

◆ Step 4: Expose Nginx Pod as Service

```
kubectl expose pod nginx --type=NodePort --port=80
```

```
root@master:~# kubectl expose pod nginx --type=NodePort --port=80
service/nginx exposed
root@master:~#
```

◆ Step 5: Check Service

```
kubectl get svc
```

Example output:

nginx NodePort 10.xx.xx.xx <none> 80:30007/TCP

```
root@master:~# kubectl expose pod nginx --type=NodePort --port=80
service/nginx exposed
root@master:~# kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes   ClusterIP   10.96.0.1     <none>        443/TCP          121m
nginx        NodePort    10.106.28.160 <none>        80:30900/TCP     30s
root@master:~#
```

◆ Step 6: Access Nginx from Browser

Open in browser:

http://<worker-ip>:<nodeport>

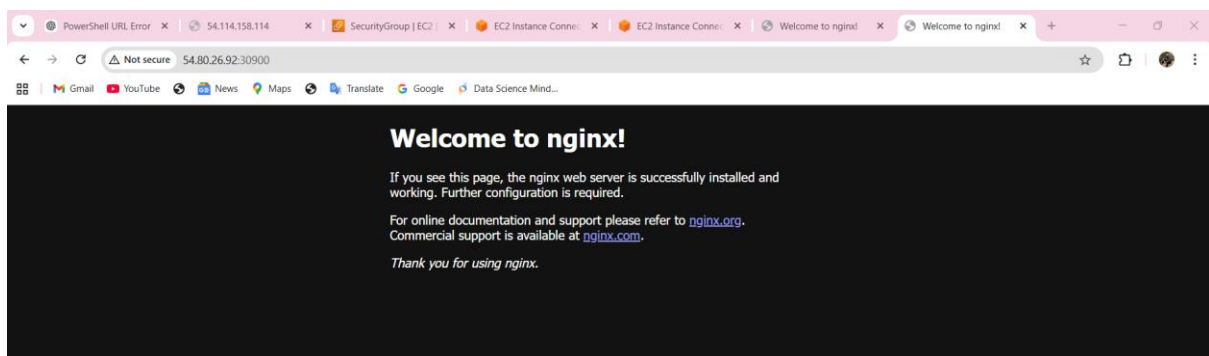
Example:

http://192.168.1.11:30007

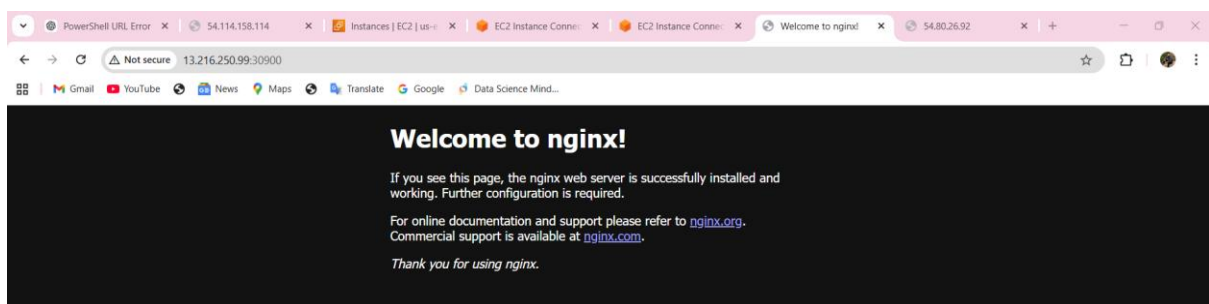
You will see:

✅ **Welcome to nginx!**

→worker-1



→worker-2



✅ **Final Verification Commands**

Check all resources

kubectl get all

```

root@master:~# kubectl get all
NAME                                READY    STATUS    RESTARTS   AGE
pod/nginx                           1/1      Running   0           2m2s
pod/nginx-deploy-6f47956ff4-5cc19   1/1      Running   0           115s
pod/nginx-deploy-6f47956ff4-64c75   1/1      Running   0           115s

NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/kubernetes                 ClusterIP           10.96.0.1        <none>            443/TCP           153m
service/nginx                       NodePort            10.106.28.160    <none>            80:30900/TCP      31m
service/nginx-deploy               NodePort            10.104.55.240    <none>            80:32655/TCP      114s

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/nginx-deploy        2/2      2              2            115s

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/nginx-deploy-6f47956ff4 2          2          2        115s
root@master:~#

```

Check cluster info

kubectl cluster-info

Check pod logs

kubectl logs nginx

```

root@master:~# kubectl cluster-info
Kubernetes control plane is running at https://172.31.32.135:6443
CoreDNS is running at https://172.31.32.135:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
root@master:~# kubectl logs nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/16 15:22:25 [notice] 1#1: using the "epoll" event method
2026/02/16 15:22:25 [notice] 1#1: nginx/1.29.5
2026/02/16 15:22:25 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/16 15:22:25 [notice] 1#1: OS: Linux 6.14.0-1018-aws
2026/02/16 15:22:25 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/16 15:22:25 [notice] 1#1: start worker processes
2026/02/16 15:22:25 [notice] 1#1: start worker process 29
2026/02/16 15:22:25 [notice] 1#1: start worker process 30
172.31.46.47 - - [16/Feb/2026:15:23:33 +0000] "GET / HTTP/1.1" 200 615 "-" Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 "-"
2026/02/16 15:23:33 [error] 2#29: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: no such file or directory), client: 172.31.46.47, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "54.80.26.92:30900", referer: "http://54.80.26.92:30900/"
172.31.46.47 - - [16/Feb/2026:15:23:33 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://54.80.26.92:30900/" Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 S
afari/537.36 "-"

```

✅ Cleanup Commands (Optional)

Delete nginx pod:

kubectl delete pod nginx

Delete nginx service:

kubectl delete svc nginx

```

root@master:~# kubectl delete pod nginx
pod "nginx" deleted from default namespace
root@master:~# kubectl delete svc nginx
service "nginx" deleted from default namespace

```

✅ Conclusion (For Your Document)

- Minikube was installed locally for single-node Kubernetes practice.
- Kubernetes cluster was built using kubeadm with 1 master and 2 workers.
- Nginx pod was deployed and accessed using NodePort service.

Troubleshooting Worker-1 Node NotReady Issue (Kubernetes Cluster)

◆ Problem Statement

After setting up Kubernetes master and worker nodes, **worker-1 was showing NotReady** when checked from master node.

Command used:

```
kubectl get nodes -o wide
```

Output:

```
worker-1  NotReady
```

```
worker-2  Ready
```

```
master    Ready
```

Because of this, **NodePort service was not working on worker-1 public IP.**

◆ Step 1: Verify Node Status

From master node, worker-1 was showing:

- Status: **NotReady**

So we confirmed that worker-1 is not healthy.

◆ Step 2: Check Detailed Node Issue

We ran:

```
kubectl describe node worker-1
```

In the output, the main error found was:

```
NodeStatusUnknown
```

```
Kubelet stopped posting node status
```

This indicated that the kubelet service on worker-1 was not running properly or not sending status updates to master.

◆ Step 3: Verify Network Connectivity Between Nodes

We checked if master can reach worker-1:

```
ping worker-1
```

Initially, ping was failing (100% packet loss), so we suspected AWS Security Group issue.

Later after updating Security Group inbound rules, ping started working successfully.

This confirmed that **network connectivity was fixed**.

◆ **Step 4: Check kubelet Service Logs on Worker-1**

Since the error mentioned kubelet issue, we checked logs:

```
sudo journalctl -u kubelet -n 50 --no-pager
```

We found the major error:

failed to load kubelet config file

```
/var/lib/kubelet/config.yaml: no such file or directory
```

This confirmed that kubelet was crashing continuously because kubelet configuration file was missing.

Also restart counter was high:

restart counter is at 100+

◆ **Root Cause Identified**

✅ **Root Cause:**

Worker-1 had an incomplete / corrupted kubeadm join process, due to which kubelet config file was missing:

```
/var/lib/kubelet/config.yaml
```

◆ **Step 5: Fix Worker-1 by Resetting kubeadm**

To fix this, we performed a full reset of worker-1 node.

Run on worker-1:

```
sudo kubeadm reset -f
```

Then removed old Kubernetes configs:

```
sudo rm -rf /etc/kubernetes /var/lib/kubelet /var/lib/etcd ~/.kube
```

Restarted services:

```
sudo systemctl restart containerd
```

```
sudo systemctl restart kubelet
```

◆ Step 6: Rejoin Worker-1 to the Cluster

We ran the kubeadm join command again on worker-1:

```
sudo kubeadm join 172.31.32.135:6443 --token 809ec2.rzftom0ts6wn7j75 --discovery-  
token-ca-cert-hash  
sha256:64b522427e97cd2cc50773fcf4901b66406705865501c9047609a28c1da01df8
```

This time the join process completed successfully.

◆ Step 7: Verify Worker-1 is Ready

After rejoining, we checked from master:

```
kubectl get nodes -o wide
```

Output:

```
master    Ready
```

```
worker-1  Ready
```

```
worker-2  Ready
```

So worker-1 was fixed successfully.

◆ Step 8: Validate Kubernetes Cluster Health

We confirmed cluster is running:

```
kubectl cluster-info
```

We also verified pods and system services are running properly:

```
kubectl get pods -A
```

◆ Step 9: Issue with Nginx Pod Scheduling

Initially nginx pod was running only on worker-2 because worker-1 was NotReady at that time.

So we created a deployment with 2 replicas to ensure nginx runs on both worker nodes:

```
kubectl create deployment nginx-deploy --image=nginx
```

```
kubectl scale deployment nginx-deploy --replicas=2
```

Verified pods:

```
kubectl get pods -o wide
```

Output showed nginx running on both nodes:

```
nginx-deploy-xxx worker-1
```

```
nginx-deploy-yyy worker-2
```

Final Result

Worker-1 issue resolved successfully.

- ✓ worker-1 became Ready
 - ✓ kubelet issue fixed
 - ✓ nginx pods running on both worker nodes
 - ✓ NodePort service accessible from both worker public IPs
-